



IT 6/L – Fundamentals of Database Systems

Hospital Queue Management System: St. Gabriel Medical Center

Second Term, Second Semester SY 2025-2026

Submitted by:
Christian Paul F. Erasmo

Submitted to:
Sheila Marie Demetita Villarin, MIT

Date Submitted:
May 2026

Contents

I. Project Context	3
II. Problem Statement.....	3
III. Proposed Solution	3
IV. Objectives of the Study.....	4
V. Scope and Limitations	4
VI. System Analysis and Design.....	4
VII. Database Design	8
VIII. Advanced Database Features	16
IX. System Features and Functionality	28
X. Prototype / User Interface	31
XI. Testing and Results	38
XII. Conclusion and Recommendations.....	43

I. Project Context

Hospitals such as **St. Gabriel Medical Center** often face challenges in managing large numbers of patients, especially when relying on traditional queue systems like manual logbooks or simple first-come-first-served methods. These approaches can lead to long waiting times, confusion, and overall inefficiency. In the current process, patients typically register at the reception and wait for their turn to be called, often without clear information about their queue status or expected waiting time, resulting in overcrowding and frustration. The proposed Hospital Queue System aims to address these issues by introducing an automated and digital approach to queue management, helping streamline patient flow, provide real-time updates, and improve overall service efficiency. This project is important because it enhances patient experience, reduces waiting time, and supports hospital staff in delivering more organized and effective healthcare services.

II. Problem Statement

The current system at St. Gabriel Medical Center presents several issues that affect both patients and staff. Long waiting times occur due to inefficient queue management, while patient flow becomes disorganized, especially during peak hours. The lack of real-time information about queue status and waiting times causes confusion and overcrowding in waiting areas because of unclear scheduling. In addition, staff experience difficulty in managing and monitoring patient queues, which increases the risk of errors, delays, and missed patient calls. As a result, the overall service experience is negatively affected, leading to low patient satisfaction.

III. Proposed Solution

The proposed Hospital Queue Management System for St. Gabriel Medical Center addresses the identified problems by introducing a digital and automated approach to patient queuing. The system streamlines patient flow by allowing efficient registration and organized queue allocation, reducing long waiting times and minimizing overcrowding. It provides realtime updates on queue status and estimated waiting times, keeping patients informed and improving their overall experience. Additionally, the system enables staff to easily monitor and manage queues through a centralized interface, reducing errors, missed calls, and delays. By improving efficiency, organization, and communication, the system enhances both patient satisfaction and the quality of healthcare service delivery.

IV. Objectives of the Study

The primary objective of this study is to design and develop a Hospital Queue Management System for St. Gabriel Medical Center that improves patient flow, reduces waiting time, and enhances the overall efficiency of hospital services. Specifically, it aims to analyze the current queuing process and identify existing issues, develop an automated system for patient queueing and prioritization, minimize delays through a more organized process, provide real-time updates and notifications on queue status, improve staff efficiency in handling patient flow and scheduling, ensure accurate recording and monitoring of queue data, and enhance patient satisfaction by offering a more convenient and transparent queuing experience.

V. Scope and Limitations

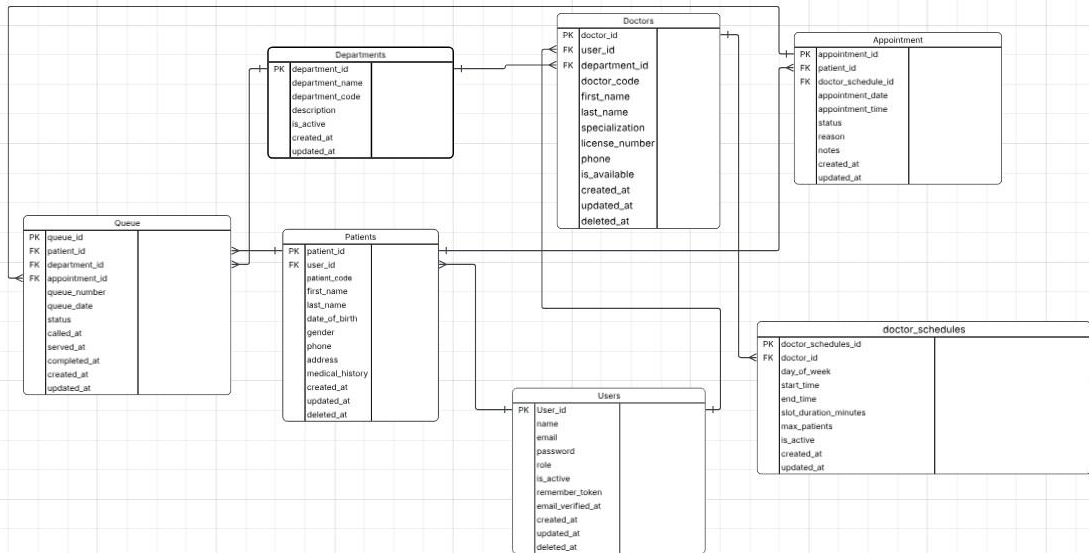
The study covers the design and development of a Hospital Queue Management System for St. Gabriel Medical Center that focuses on managing patient queues, automating queue assignment and prioritization, and providing real-time updates on queue status to both patients and staff. It includes features such as patient registration for queueing, display of queue numbers, and basic monitoring of patient flow within the facility. However, the system is limited to queue management functions and does not include full hospital information system features such as electronic medical records, billing, or advanced appointment scheduling. It is also confined to use within St. Gabriel Medical Center and depends on available hardware, network infrastructure, and user compliance for optimal performance.

VI. System Analysis and Design

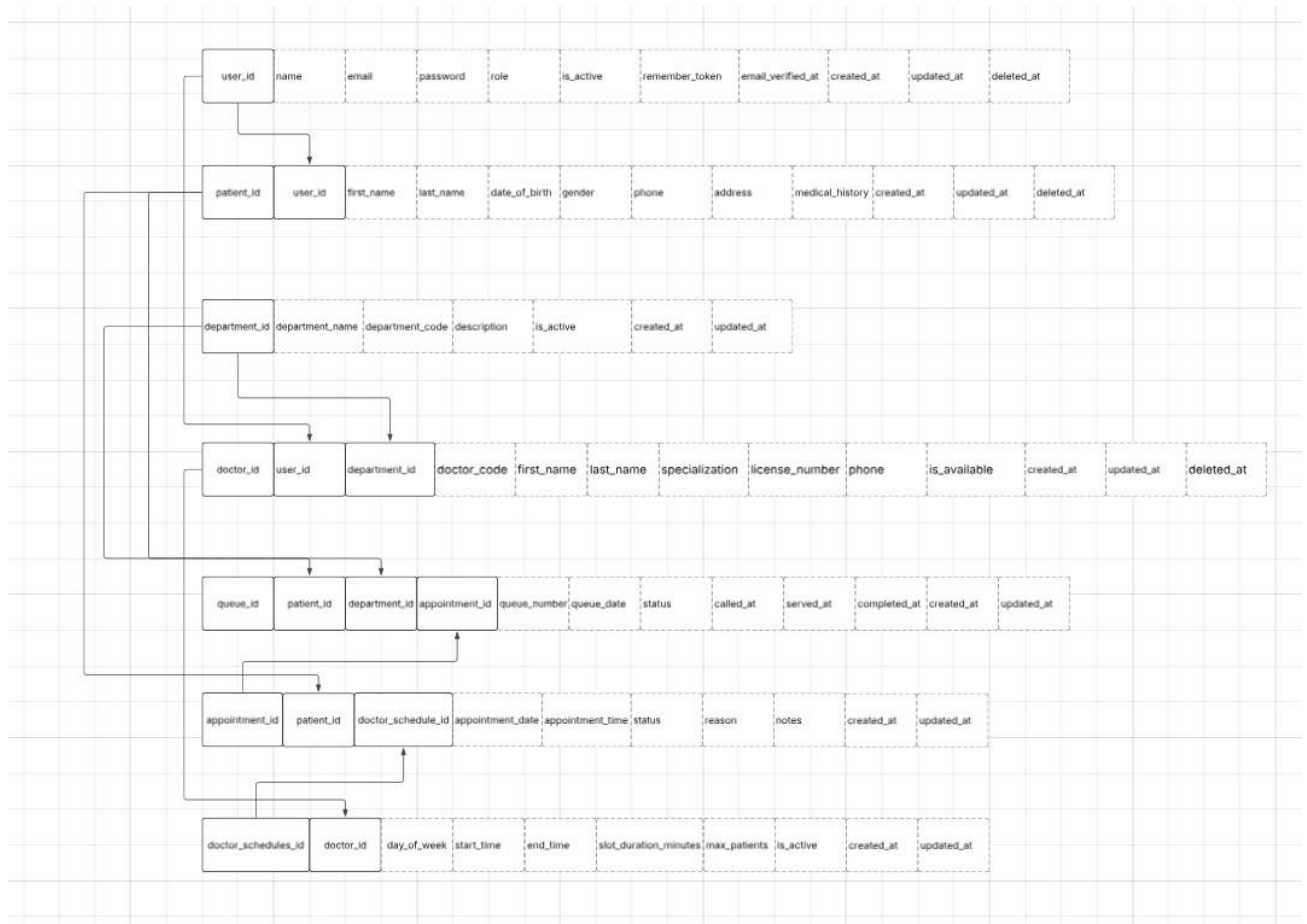
A.) Business Rules

- A patient can have multiple queue entries and appointments
- A patient can have multiple schedules through doctor bookings
- A queue entry belongs to only one patient
- A queue entry is assigned to only one department
- A queue entry can be linked to one appointment
- A department can handle multiple queue entries
- A department can have multiple doctors
- A doctor belongs to only one department
- A doctor can have multiple schedules across different days
- A doctor schedule can accommodate multiple appointments up to its maximum patient limit
- An appointment belongs to only one patient and one doctor schedule
- Each appointment is linked to one queue entry
- A user account can be associated with either a doctor or a patient, but not both
- Only authorized users (Admin, Staff, Patient) can access and update system data based on their assigned role

B.) ERD



C.) Explanation of entities, relationships, and processes



- One user account can belong to one patient
- One user account can belong to one doctor
- One department can have many doctors
- One doctor can have many schedules
- One patient can book many appointments
- One doctor schedule can accommodate many appointments
- One patient can have many queue records
- One department can manage many queue entries
- One appointment can generate queue transactions
- One patient can be assigned to different departments through appointments
- One doctor belongs to one department
- One queue record belongs to one patient
- One queue record may be linked to one appointment

- One appointment belongs to one patient
- One appointment belongs to one doctor schedule
- One doctor schedule belongs to one doctor

VII. Database Design

a.) List of Tables

Filters

Containing the word:

Table	Action	Rows	Type	Collation	Size	Overhead
☐ appointments		0	InnoDB	utf8mb4_unicode_ci	96.0 K1B	-
☐ archived_patients		0	InnoDB	utf8mb4_unicode_ci	16.0 K1B	-
☐ cache		0	InnoDB	utf8mb4_unicode_ci	32.0 K1B	-
☐ cache_locks		0	InnoDB	utf8mb4_unicode_ci	32.0 K1B	-
☐ departments		5	InnoDB	utf8mb4_unicode_ci	32.0 K1B	-
☐ doctors		4	InnoDB	utf8mb4_unicode_ci	80.0 K1B	-
☐ doctor_schedules		0	InnoDB	utf8mb4_unicode_ci	48.0 K1B	-
☐ failed_jobs		0	InnoDB	utf8mb4_unicode_ci	32.0 K1B	-
☐ jobs		0	InnoDB	utf8mb4_unicode_ci	32.0 K1B	-
☐ job_batches		0	InnoDB	utf8mb4_unicode_ci	16.0 K1B	-
☐ migrations		17	InnoDB	utf8mb4_unicode_ci	16.0 K1B	-
☐ model_names		0	InnoDB	utf8mb4_unicode_ci	16.0 K1B	-
☐ password_reset_tokens		0	InnoDB	utf8mb4_unicode_ci	16.0 K1B	-
☐ patients		2	InnoDB	utf8mb4_unicode_ci	48.0 K1B	-
☐ personal_access_tokens		0	InnoDB	utf8mb4_unicode_ci	64.0 K1B	-
☐ queues		12	InnoDB	utf8mb4_unicode_ci	64.0 K1B	-
☐ sessions		2	InnoDB	utf8mb4_unicode_ci	48.0 K1B	-
☐ users		11	InnoDB	utf8mb4_unicode_ci	48.0 K1B	-
☐ hospital_queue		0	InnoDB	utf8mb4_unicode_ci	16.0 K1B	-
10 tables	Sum	59	InnoDB	utf8mb4_general_ci	736.0 K1B	0 B

Note: The Tables that are highlighted in the list such as the cache, sessions, jobs, migrations, and the password_reset_tokens are Laravel framework system tables and are not part of the core application design.

b.) Fields and Data Types

Users Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
☐ 2	name	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
☐ 3	email	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
☐ 4	role	enum('admin', 'staff', 'patient')	utf8mb4_unicode_ci		No	patient			Change Drop More
☐ 5	is_active	tinyint(1)			No	1			Change Drop More
☐ 6	email_verified_at	timestamp			Yes	NULL			Change Drop More
☐ 7	password	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
☐ 8	remember_token	varchar(100)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
☐ 9	created_at	timestamp			Yes	NULL			Change Drop More
☐ 10	updated_at	timestamp			Yes	NULL			Change Drop More
☐ 11	deleted_at	timestamp			Yes	NULL			Change Drop More

Departments Table

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	id 	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	 Change  Drop  More
☐	2	name	varchar(255)	utf8mb4_unicode_ci		No	None			 Change  Drop  More
☐	3	code 	varchar(255)	utf8mb4_unicode_ci		No	None			 Change  Drop  More
☐	4	description	text	utf8mb4_unicode_ci		Yes	NULL			 Change  Drop  More
☐	5	is_active	tinyint(1)			No	1			 Change  Drop  More
☐	6	created_at	timestamp			Yes	NULL			 Change  Drop  More
☐	7	updated_at	timestamp			Yes	NULL			 Change  Drop  More

Patients Table

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	id 	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	 Change  Drop  More
☐	2	user_id 	bigint(20)		UNSIGNED	No	None			 Change  Drop  More
☐	3	patient_code 	varchar(255)	utf8mb4_unicode_ci		No	None			 Change  Drop  More
☐	4	first_name	varchar(255)	utf8mb4_unicode_ci		No	None			 Change  Drop  More
☐	5	last_name	varchar(255)	utf8mb4_unicode_ci		No	None			 Change  Drop  More
☐	6	date_of_birth	date			No	None			 Change  Drop  More
☐	7	gender	enum('male', 'female', 'other')	utf8mb4_unicode_ci		No	None			 Change  Drop  More
☐	8	phone	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			 Change  Drop  More
☐	9	address	text	utf8mb4_unicode_ci		Yes	NULL			 Change  Drop  More
☐	10	medical_history	text	utf8mb4_unicode_ci		Yes	NULL			 Change  Drop  More
☐	11	created_at	timestamp			Yes	NULL			 Change  Drop  More
☐	12	updated_at	timestamp			Yes	NULL			 Change  Drop  More
☐	13	deleted_at	timestamp			Yes	NULL			 Change  Drop  More

Doctors Table

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	id 	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	 Change  Drop  More
☐	2	user_id 	bigint(20)		UNSIGNED	No	None			 Change  Drop  More
☐	3	department_id 	bigint(20)		UNSIGNED	No	None			 Change  Drop  More
☐	4	doctor_code 	varchar(255)	utf8mb4_unicode_ci		No	None			 Change  Drop  More
☐	5	first_name	varchar(255)	utf8mb4_unicode_ci		No	None			 Change  Drop  More
☐	6	last_name	varchar(255)	utf8mb4_unicode_ci		No	None			 Change  Drop  More
☐	7	specialization	varchar(255)	utf8mb4_unicode_ci		No	None			 Change  Drop  More
☐	8	license_number 	varchar(255)	utf8mb4_unicode_ci		No	None			 Change  Drop  More
☐	9	phone	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			 Change  Drop  More
☐	10	is_available	tinyint(1)			No	1			 Change  Drop  More
☐	11	created_at	timestamp			Yes	NULL			 Change  Drop  More
☐	12	updated_at	timestamp			Yes	NULL			 Change  Drop  More
☐	13	deleted_at	timestamp			Yes	NULL			 Change  Drop  More

Doctor_Schedules Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
☐ 2	doctor_id	bigint(20)		UNSIGNED	No	None			Change Drop More
☐ 3	day_of_week	tinyint(4)			No	None			Change Drop More
☐ 4	start_time	time			No	None			Change Drop More
☐ 5	end_time	time			No	None			Change Drop More
☐ 6	slot_duration_minutes	int(11)			No	30			Change Drop More
☐ 7	max_patients	int(11)			No	10			Change Drop More
☐ 8	is_active	tinyint(1)			No	1			Change Drop More
☐ 9	created_at	timestamp			Yes	NULL			Change Drop More
☐ 10	updated_at	timestamp			Yes	NULL			Change Drop More

Appointments Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
☐ 2	patient_id	bigint(20)		UNSIGNED	No	None			Change Drop More
☐ 3	doctor_id	bigint(20)		UNSIGNED	No	None			Change Drop More
☐ 4	doctor_schedule_id	bigint(20)		UNSIGNED	No	None			Change Drop More
☐ 5	appointment_date	date			No	None			Change Drop More
☐ 6	appointment_time	time			No	None			Change Drop More
☐ 7	status	enum('pending', 'confirmed', 'completed', 'canceled', 'cancelled')	utf8mb4_unicode_ci		No	pending			Change Drop More
☐ 8	reason	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
☐ 9	notes	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
☐ 10	reference_code	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
☐ 11	created_at	timestamp			Yes	NULL			Change Drop More
☐ 12	updated_at	timestamp			Yes	NULL			Change Drop More

Queues Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
☐ 2	patient_id	bigint(20)		UNSIGNED	No	None			Change Drop More
☐ 3	department_id	bigint(20)		UNSIGNED	No	None			Change Drop More
☐ 4	appointment_id	bigint(20)		UNSIGNED	Yes	NULL			Change Drop More
☐ 5	queue_number	int(11)			No	None			Change Drop More
☐ 6	queue_code	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
☐ 7	queue_date	date			No	None			Change Drop More
☐ 8	status	enum('waiting', 'serving', 'done', 'skipped')	utf8mb4_unicode_ci		No	waiting			Change Drop More
☐ 9	called_at	timestamp			Yes	NULL			Change Drop More
☐ 10	served_at	timestamp			Yes	NULL			Change Drop More
☐ 11	done_at	timestamp			Yes	NULL			Change Drop More
☐ 12	created_at	timestamp			Yes	NULL			Change Drop More
☐ 13	updated_at	timestamp			Yes	NULL			Change Drop More

C.) Constraints (Primary Key, Foreign Key, etc.)

The Following constraints are applied across the CMS Database:

Constraint	Tables Applied	Description
PRIMARY KEY (PK)	All tables	Each table has an auto-incrementing integer as its primary key
FOREIGN KEY (FK)	patients, doctors, doctor_schedules, appointments, queues, archived_patients	Foreign keys enforce referential integrity between related tables — patients and doctors reference users, appointments reference patients and doctors, queues reference patients and departments
UNIQUE	users.email, doctors.license_number, patients.patient_code	Prevents duplicate email addresses, license numbers, and patient codes across the system
NOT NULL	All PK and FK columns, required form fields	Ensures critical fields such as patient_code, first_name, last_name, email, role, queue_date, and appointment_date always contain a value
SOFT DELETE	users, patients, doctors	Uses deleted_at timestamp tracking instead of permanently removing records, allowing deleted accounts to be restored or archived at any time
CASCADE DELETE	archived_patients	When a patient is permanently archived, all associated appointment and queue history snapshots are stored in the archived_patients table before the original records are removed
DEFAULT	appointments.status, queues.status, users.is_active	Provides default values for status columns — appointments default to pending, queues default to waiting, and user accounts default to active
ENUM	appointments.status, queues.status, users.role, patients.gender	Restricts column values to a predefined set of valid options — appointment statuses are limited to pending, confirmed, completed, and canceled; queue statuses are limited to waiting, serving, done, and skipped; user roles are limited to admin, staff, and patient; gender is limited to male, female, and other
CHECK / TRIGGER	doctor_schedules.end_time, appointments, queues	Database-level triggers enforce business rules such as end time must be after start time, no double booking of the same doctor slot, no duplicate active queue entries for the same patient and department on the same day, and prevention of status changes on completed appointments

Constraint Tables

Table	Column	Constraint	Reference
users	id	PRIMARY KEY	-
users	email	UNIQUE	-
users	role	ENUM	admin, staff, patient
users	deleted_at	SOFT DELETE	-
departments	id	PRIMARY KEY	-
departments	name	NOT NULL	-
patients	id	PRIMARY KEY	-
patients	user_id	FOREIGN KEY	users.id
patients	patient_code	UNIQUE, NOT NULL	-
patients	gender	ENUM	male, female, other
patients	deleted_at	SOFT DELETE	-
doctors	id	PRIMARY KEY	-
doctors	user_id	FOREIGN KEY	users.id
doctors	department_id	FOREIGN KEY	departments.id
doctors	license_number	UNIQUE, NOT NULL	-
doctors	deleted_at	SOFT DELETE	-
doctor_schedules	id	PRIMARY KEY	-
doctor_schedules	doctor_id	FOREIGN KEY	doctors.id
doctor_schedules	day_of_week	NOT NULL	0=Sun ... 6=Sat
doctor_schedules	end_time	TRIGGER CHECK	Must be after start_time
doctor_schedules	max_patients	TRIGGER CHECK	Must be greater than 0
appointments	id	PRIMARY KEY	-
appointments	patient_id	FOREIGN KEY	patients.id
appointments	doctor_id	FOREIGN KEY	doctors.id
appointments	doctor_schedule_id	FOREIGN KEY	doctor_schedules.id
appointments	reference_code	UNIQUE, NOT NULL	-
appointments	status	ENUM, DEFAULT	pending, confirmed, completed, canceled
appointments	appointment_date + appointment_time + doctor_id	TRIGGER CHECK	No double booking allowed
queues	id	PRIMARY KEY	-
queues	patient_id	FOREIGN KEY	patients.id
queues	department_id	FOREIGN KEY	departments.id
queues	appointment_id	FOREIGN KEY	appointments.id
queues	status	ENUM, DEFAULT	waiting, serving, done, skipped
queues	patient_id + department_id + queue_date	TRIGGER CHECK	No duplicate active queue per patient per department per day
archived_patients	id	PRIMARY KEY	-
archived_patients	original_patient_id	NOT NULL	patients.id (reference only)
archived_patients	original_user_id	NOT NULL	users.id (reference only)
archived_patients	appointments_snapshot	JSON	Full appointment history snapshot
archived_patients	queues_snapshot	JSON	Full queue history snapshot
archived_patients	archived_at	NOT NULL	-

Foreign Key Constraints (phpMyAdmin)

Patient Constraint

Foreign key constraints

Actions	Constraint properties	Column	Foreign key constraint (INNODB)		
			Database	Table	Column
Drop	patients_user_id_foreign ON DELETE CASCADE ON UPDATE RESTRICT	user_id	hospital_qms	users	id
		+ Add column			
	Constraint name ON DELETE RESTRICT ON UPDATE RESTRICT		hospital_qms		
		+ Add column			
+ Add constraint					

Doctor Constraint

Foreign key constraints

Actions	Constraint properties	Column	Foreign key constraint (INNODB)		
			Database	Table	Column
Drop	doctors_department_id_forei ON DELETE RESTRICT ON UPDATE RESTRICT	department_id	hospital_qms	departments	id
		+ Add column			
Drop	doctors_user_id_foreign ON DELETE CASCADE ON UPDATE RESTRICT	user_id	hospital_qms	users	id
		+ Add column			
	Constraint name ON DELETE RESTRICT ON UPDATE RESTRICT		hospital_qms		
		+ Add column			
+ Add constraint					

Doctor_Schedules Constraint

Foreign key constraints

Actions	Constraint properties	Column	Foreign key constraint (INNODB)		
			Database	Table	Column
Drop	doctor_schedules_doctor_id_ ON DELETE CASCADE ON UPDATE RESTRICT	doctor_id	hospital_qms	doctors	id
		+ Add column			
	Constraint name ON DELETE RESTRICT ON UPDATE RESTRICT		hospital_qms		
		+ Add column			
+ Add constraint					

Appointments Constraint

Foreign key constraints

Actions	Constraint properties	Column	Foreign key constraint (INNODB)		
			Database	Table	Column
Drop	appointments_doctor_id_fore ON DELETE CASCADE ON UPDATE RESTRICT	doctor_id	hospital_qms	doctors	id
		+ Add column			
Drop	appointments_doctor_schedi ON DELETE RESTRICT ON UPDATE RESTRICT	doctor_schedule_id	hospital_qms	doctor_schedules	id
		+ Add column			
Drop	appointments_patient_id_fon ON DELETE CASCADE ON UPDATE RESTRICT	patient_id	hospital_qms	patients	id
		+ Add column			
	Constraint name ON DELETE RESTRICT ON UPDATE RESTRICT		hospital_qms		
		+ Add column			
+ Add constraint					

Queue Constraint

Foreign key constraints

Actions	Constraint properties	Column	Foreign key constraint (INNODB)		
			Database	Table	Column
Drop	queues_appointment_id_forei	ON DELETE SET NULL ON UPDATE RESTRICT	appointment_id	hospital_qms	appointments id
		+ Add column			
Drop	queues_department_id_forei	ON DELETE CASCADE ON UPDATE RESTRICT	department_id	hospital_qms	departments id
		+ Add column			
Drop	queues_patient_id_foreign	ON DELETE CASCADE ON UPDATE RESTRICT	patient_id	hospital_qms	patients id
		+ Add column			
	Constraint name	ON DELETE RESTRICT ON UPDATE RESTRICT		hospital_qms	
		+ Add column			
+ Add constraint					

VIII. Advanced Database Features

Triggers

Trigger 1: trg_after_patient_delete

Purpose: The foreign key constraints in the appointments table automatically enforce data consistency whenever records are added, updated, or deleted across related tables. Its purpose is to ensure that every appointment always references a real and existing doctor, schedule slot, and patient preventing corrupt or incomplete records from being stored in the database.

Explanation: The table has three foreign key constraints. The doctor_id column references doctors.id with CASCADE on delete, meaning if a doctor is removed, all their appointments are automatically deleted as well. The doctor_schedule_id column references doctor_schedules.id with RESTRICT on delete, which blocks deletion of a schedule slot if an appointment is still using it — preventing broken bookings. The patient_id column references patients.id with CASCADE on delete, so removing a patient also removes all their associated appointments. Together, these constraints enforce referential integrity across the system, ensuring no appointment can exist without a corresponding doctor, schedule, or patient record.

Edit

Details

Trigger name

trg_after_patient_delete

Table

patients

Time

AFTER

Event

DELETE

Definition

```
1 BEGIN
2                                     -- Cancel all future pending/confirmed
appointments
3                                     UPDATE appointments
4                                     SET status = "canceled"
5                                     WHERE patient_id = OLD.id
6                                     AND status IN ("pending", "confirmed")
7                                     AND appointment_date >= CURDATE();
8
9                                     -- Mark any active queue entries as
skipped
10
11                                     UPDATE queues
12                                     SET status = "skipped",
13                                     done_at = NOW()
14                                     WHERE patient_id = OLD.id
```

Definer

root@localhost

Go

Close

Trigger 2: trg_after_queue_done

Purpose: The trg_after_queue_done trigger is designed to automatically mark a linked appointment as completed whenever a queue entry is updated to a done status. Its purpose is to keep the appointments table in sync with the queues table without requiring any manual update or additional process from the staff.

Explanation: This trigger fires after an UPDATE is made on the queues table. It checks whether the queue's new status is "done" and the old status was not yet "done", and that the queue entry has a linked appointment via appointment_id. If all those conditions are met, it automatically updates the corresponding record in the appointments table, setting its status to "completed" but only if that appointment was still in a "pending" or "confirmed" state. This ensures that once a patient's queue session is finished, their appointment status is reflected as completed in real time, maintaining consistency across both tables automatically.

Edit

Details

Trigger name

trg_after_queue_done

Table

queues

Time

AFTER

Event

UPDATE

Definition

```
1 BEGIN
2     -- When queue is marked done and has a
    linked appointment
3     IF NEW.status = "done" AND OLD.status !=
    "done"
4         AND NEW.appointment_id IS NOT NULL THEN
5         UPDATE appointments
6         SET status = "completed"
7         WHERE id = NEW.appointment_id
8             AND status IN ("pending",
    "confirmed");
9         END IF;
10    END
```

Definer

root@localhost

Go

Close

Trigger 3: trg_after_queue_insert

Purpose: The trg_after_queue_insert trigger is designed to automatically confirm a linked appointment the moment a patient joins the queue. Its purpose is to eliminate the need for staff to manually update an appointment's status when a patient physically arrives and enters the waiting queue.

Explanation: This trigger fires after a new row is inserted into the queues table. It checks whether the newly created queue entry has a status of "waiting" and is linked to an existing appointment via appointment_id. If both conditions are met, it automatically updates the corresponding record in the appointments table, changing its status from "pending" to "confirmed". This ensures that as soon as a patient is added to the queue, their appointment is instantly recognized as confirmed keeping the queues and appointments tables synchronized in real time without any additional manual input from the staff.

Edit

Details

Trigger name

trg_after_queue_insert

Table

queues

Time

AFTER

Event

INSERT

Definition

```
1 BEGIN
2      -- Only act when a new waiting queue entry
   is created
3      IF NEW.status = "waiting" AND
NEW.appointment_id IS NOT NULL THEN
4      -- Auto-confirm the linked appointment
   when patient joins queue
5      UPDATE appointments
6      SET status = "confirmed"
7      WHERE id = NEW.appointment_id
8      AND status = "pending";
9      END IF;
10     END
```

Definer

root@localhost

Go

Close

Trigger 4: trg_before_appointment_insert

Purpose: The trg_before_appointment_insert trigger is designed to automatically prevent double-booking a doctor at the same date and time before a new appointment is even saved. Its purpose is to protect the integrity of the scheduling system by blocking any conflicting appointment from being inserted into the database.

Explanation: This trigger fires before a new row is inserted into the appointments table. It first counts how many existing appointments share the same doctor_id, appointment_date, and appointment_time as the one being inserted, excluding any that are already "canceled". If the count is greater than zero, meaning a conflict exists, it raises a custom SQL error using SIGNAL SQLSTATE '45000' with a message indicating that the appointment slot is already taken and the insert is blocked entirely. This ensures that no two active appointments can ever be booked for the same doctor at the same time, enforcing schedule conflict prevention automatically at the database level without relying on application-side validation alone.

Edit

Details

Trigger name

trg_before_appointment_ir

Table

appointments

Time

BEFORE

Event

INSERT

Definition

```
1 BEGIN
2     DECLARE conflict_count INT;
3
4     SELECT COUNT(*) INTO conflict_count
5     FROM appointments
6     WHERE doctor_id      = NEW.doctor_id
7         AND appointment_date =
8     NEW.appointment_date
9         AND appointment_time =
10    NEW.appointment_time
11         AND status       != "canceled";
12
13     IF conflict_count > 0 THEN
14         SIGNAL SQLSTATE "45000"
15         SET MESSAGE_TEXT = "This appointment
```

Definer

root@localhost

Go

Close

Trigger 5: trg_before_appointment_update

Purpose: The trg_before_appointment_update trigger is designed to automatically prevent any status changes from being made to an appointment that has already been marked as completed. Its purpose is to protect finalized appointment records from being accidentally or intentionally modified, preserving the accuracy of the system's historical data.

Explanation: This trigger fires before any UPDATE is made on the appointments table. It checks whether the old status of the appointment was "completed" and the new status being set is anything other than "completed". If that condition is true, it raises a custom SQL error using SIGNAL SQLSTATE '45000' with the message "Cannot change the status of a completed appointment." and the update is blocked entirely. This ensures that once an appointment has been marked as completed, its status becomes permanent and cannot be overwritten or reversed, safeguarding the integrity of completed records at the database level.

Edit

Details

Trigger name

trg_before_appointment_u

Table

appointments

Time

BEFORE

Event

UPDATE

Definition

```
1 BEGIN
2     IF OLD.status = "completed" AND NEW.status
3     != "completed" THEN
4         SIGNAL SQLSTATE "45000"
5         SET MESSAGE_TEXT = "Cannot change the
6         status of a completed appointment.";
7     END IF;
8     END
```

Definer

root@localhost

Go

Close

Trigger 6: trg_before_patient_insert

Purpose: The trg_before_patient_insert trigger is designed to automatically validate that every new patient record has a patient code before it is saved to the database. Its purpose is to ensure that no patient can ever be registered in the system without a proper identifying code, maintaining the completeness and reliability of patient records.

Edit

Details

Trigger name

trg_before_patient_insert

Table

patients

Time

BEFORE

Event

INSERT

Definition

```
1 BEGIN
2     IF NEW.patient_code IS NULL OR
3     NEW.patient_code = "" THEN
4         SIGNAL SQLSTATE "45000"
5         SET MESSAGE_TEXT = "Patient code cannot
6         be empty.";
7     END IF;
8 END
```

Definer

root@localhost

Go

Close

Trigger 7: trg_before_queue_insert

Purpose: The trg_before_queue_insert trigger is designed to automatically prevent a patient from being added to the same department's queue more than once on the same day. Its purpose is to ensure that no duplicate active queue entries exist for a patient within the same department, keeping the queue system fair and organized.

Explanation: This trigger fires before a new row is inserted into the queues table. It first counts how many existing queue entries share the same patient_id, department_id, and queue_date as the one being inserted, and where the status is either "waiting" or "serving". If the count is greater than zero, meaning the patient already has an active queue entry in that department for that day, it raises a custom SQL error using SIGNAL SQLSTATE '45000' with the message "Patient already has an active queue entry in this department today." and the insert is blocked entirely. This ensures that a patient cannot queue twice in the same department on the same day, preventing duplicate entries and maintaining the integrity of the queue at the database level.

Edit

Details

Trigger name

trg_before_queue_insert

Table

queues

Time

BEFORE

Event

INSERT

Definition

1 BEGIN

2 DECLARE existing_count INT;

3

4 SELECT COUNT(*) INTO existing_count

5 FROM queues

6 WHERE patient_id = NEW.patient_id

7 AND department_id = NEW.department_id

8 AND queue_date = NEW.queue_date

9 AND status IN ("waiting",

10 "serving");

11

12 IF existing_count > 0 THEN

13 SIGNAL SQLSTATE "45000"

SET MESSAGE_TEXT = "Patient already

has an active queue entry in this department today.";

Definer

root@localhost

Go

Close

Trigger 8: trg_before_schedule_insert

Purpose: The trg_before_schedule_insert trigger is designed to automatically validate a doctor's schedule entry before it is saved to the database. Its purpose is to ensure that every schedule record has a logical time range and a valid patient capacity, preventing incorrectly configured schedules from ever being created in the system.

Explanation: This trigger fires before a new row is inserted into the doctor_schedules table. It performs two separate validations. First, it checks whether the end_time is less than or equal to the start_time if so, it raises a custom SQL error with the message "Schedule end time must be after start time." and blocks the insert. Second, it checks whether max_patients is less than or equal to zero if so, it raises another error with the message "Max patients must be greater than zero." and blocks the insert as well. This ensures that all doctor schedules stored in the system always have a valid time range and a meaningful patient limit, enforcing data correctness at the database level before any invalid record can be saved.

Edit

Details

Trigger name

trg_before_schedule_insert

Table

doctor_schedules

Time

BEFORE

Event

INSERT

Definition

```
1 BEGIN
2     IF NEW.end_time <= NEW.start_time THEN
3         SIGNAL SQLSTATE "45000"
4         SET MESSAGE_TEXT = "Schedule end time
must be after start time.";
5     END IF;
6
7     IF NEW.max_patients <= 0 THEN
8         SIGNAL SQLSTATE "45000"
9         SET MESSAGE_TEXT = "Max patients must be
greater than zero.";
10    END IF;
11    END
```

Definer

root@localhost

Go

Close

Trigger 9: trg_before_schedule_update

Purpose: The trg_before_schedule_update trigger is designed to automatically validate any changes made to a doctor's schedule before they are saved to the database. Its purpose is to ensure that an updated schedule always maintains a logical time range, preventing invalid time configurations from being applied to an existing schedule record.

Explanation: This trigger fires before any UPDATE is made on the doctor_schedules table. It checks whether the updated end_time is less than or equal to the updated start_time. If that condition is true, it raises a custom SQL error using SIGNAL SQLSTATE '45000' with the message "Schedule end time must be after start time." and the update is blocked entirely. This ensures that whenever a doctor's schedule is modified, the end time must always come after the start time, enforcing time validity at the database level and preventing any illogical or broken schedule entries from being saved through an update.

Edit

Details

Trigger name

trg_before_schedule_updat

Table

doctor_schedules

Time

BEFORE

Event

UPDATE

Definition

1 BEGIN

2 IF NEW.end_time <= NEW.start_time THEN

3 SIGNAL SQLSTATE "45000"

4 SET MESSAGE_TEXT = "Schedule end time

must be after start time.";

5 END IF;

6 END

Definer

root@localhost

Go

Close

Views

View 1 – appointment_status_report:

Provides a consolidated look at each patient's personal details and their appointment history, including the assigned doctor and department. Ideal for front desk staff who need a quick snapshot of a patient's records.

Server: 127.0.0.1 » Database: hospital_qms » View: appointment_status_report

Browse Structure SQL Search Insert Export Privileges Operations

Run SQL query/queries on table hospital_qms.appointment_status_report:

```

1 CREATE OR REPLACE VIEW appointment_status_report AS
2 SELECT
3     a.id AS appointment_id,
4     a.reference_code,
5     a.appointment_date,
6     a.appointment_time,
7     a.status,
8     a.reason,
9     a.notes,
10    CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
11    p.patient_code,
12    pu.email AS patient_email,
13    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
14    d.specialization,
15    dept.name AS department_name,
16    DATEDIFF(CURDATE(), a.appointment_date) AS days_since_appointment,
17    CASE
18        WHEN a.status = 'completed' THEN 'Done'
19        WHEN a.status = 'canceled' THEN 'Cancelled'
20        WHEN a.status = 'confirmed'
21            AND a.appointment_date >= CURDATE() THEN 'Confirmed'
22        WHEN a.status = 'pending'
23            AND a.appointment_date >= CURDATE() THEN 'Upcoming'
24        WHEN a.appointment_date < CURDATE() THEN 'Overdue'
25        ELSE 'Unknown'
26    END AS appointment_label
27 FROM
28     appointments a
29 JOIN
30     patients p ON p.id = a.patient_id
31 JOIN
32     users pu ON pu.id = p.user_id
33 JOIN
34     doctors d ON d.id = a.doctor_id
35 LEFT JOIN
36     departments dept ON dept.id = d.department_id
37 WHERE
38     p.deleted_at IS NULL
39 ORDER BY
40     a.appointment_date DESC;

```

Browse Result:

Browse Structure SQL Search Insert Export Privileges Operations

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds)

SELECT * FROM `appointment\_status\_report`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

appointment_id	reference_code	appointment_date	appointment_time	status	reason	notes	patient_name	patient_code	patient_email	doctor_name	specialization	department_name	days_since_appointment	appointment_label
----------------	----------------	------------------	------------------	--------	--------	-------	--------------	--------------	---------------	-------------	----------------	-----------------	------------------------	-------------------

Query results operations

Create view

View 2 – doctor_schedule_availability:

Shows each doctor's weekly schedule alongside a count of how many appointments are booked per slot. Useful for scheduling staff to quickly identify available or overbooked doctors.

```

1 CREATE OR REPLACE VIEW doctor_schedule_availability AS
2 SELECT
3     d.id                                AS doctor_id,
4     d.doctor_code,
5     CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
6     u.email                             AS doctor_email,
7     d.specialization,
8     d.license_number,
9     d.is_available                       AS doctor_is_available,
10    dept.name                            AS department_name,
11    dept.code                            AS department_code,
12    ds.id                                AS schedule_id,
13    ds.day_of_week,
14    ds.start_time,
15    ds.end_time,
16    ds.slot_duration_minutes,
17    ds.max_patients,
18    ds.is_active                          AS slot_is_active,
19    COUNT(a.id)                          AS appointments_booked
20 FROM    doctors      d
21 JOIN    users        u ON u.id          = d.user_id
22 JOIN    departments  dept ON dept.id     = d.department_id
23 LEFT JOIN doctor_schedules ds ON ds.doctor_id = d.id
24 LEFT JOIN appointments a ON a.doctor_schedule_id = ds.id
25 WHERE   d.deleted_at IS NULL
26 GROUP BY
27     d.id, d.doctor_code, d.first_name, d.last_name,
28     u.email, d.specialization, d.license_number, d.is_available,
29     dept.name, dept.code,
30     ds.id, ds.day_of_week, ds.start_time, ds.end_time,
31     ds.slot_duration_minutes, ds.max_patients, ds.is_active
32 ORDER BY d.last_name, ds.day_of_week;

```

Browse Result:

Showing rows 0 - 2 (3 total. Query took 0.0005 seconds.)

SELECT * FROM "doctor_schedule_availability"

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

doctor_id	doctor_code	doctor_name	doctor_email	specialization	license_number	doctor_is_available	department_name	department_code	schedule_id	day_of_week	start_time	end_time	slot_duration_minutes	max_patients
6	DOC-004	Ivan Entrina	ivanentrina@gmail.com	Electrophysiologist	00000000000000	1	Cardiology	CAR	NULL	NULL	NULL	NULL	NULL	NULL
7	DOC-005	Celine Erasmo	celine@gmail.com	Diagnosing	98765432123	1	Orthopedics	ORT	NULL	NULL	NULL	NULL	NULL	NULL
1	DOC-001	Maria Santos	dr.santos@hospital.com	General Practitioner	LIC-2024-001	1	General Medicine	GEN	NULL	NULL	NULL	NULL	NULL	NULL

View 3 – patient_overview:

Summarizes all appointments with a computed label (Upcoming, Confirmed, Done, Cancelled, Overdue) based on status and date. Great for admins who need a real-time overview of appointment activity across the hospital.

Run SQL query/queries on table hospital_qms.patient_overview: 

```

1 CREATE OR REPLACE VIEW patient_overview AS
2 SELECT
3     p.id                                AS patient_id,
4     p.patient_code,
5     CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
6     p.gender,
7     p.date_of_birth,
8     p.phone                             AS patient_phone,
9     u.email                             AS patient_email,
10    a.id                                AS appointment_id,
11    a.reference_code,
12    a.appointment_date,
13    a.appointment_time,
14    a.status                             AS appointment_status,
15    a.reason,
16    CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
17    d.specialization,
18    dept.name                            AS department_name
19 FROM    patients      p
20 JOIN    users          u    ON u.id      = p.user_id
21 LEFT JOIN appointments a    ON a.patient_id = p.id
22 LEFT JOIN doctors      d    ON d.id      = a.doctor_id
23 LEFT JOIN departments dept ON dept.id    = d.department_id
24 WHERE   p.deleted_at IS NULL
25 ORDER BY a.appointment_date DESC;

```

Browse Result:

Showing rows 0 - 3 (4 total, Query took 0.0441 seconds.)

SELECT * FROM `patient\_overview`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all Number of rows: 25 Filter rows:

Extra options

	patient_id	patient_code	patient_name	gender	date_of_birth	patient_phone	patient_email	appointment_id	reference_code	appointment_date	appointment_time	appointment_status	reason	doctor_n
☐ Edit Copy Delete	1	PAT-0001	kaiser Dela Cruz	male	1990-05-15	09181234567	juan@example.com	NULL	NULL	NULL	NULL	NULL	NULL	NULL
☐ Edit Copy Delete	7	PAT-0002	Jonroy Gapusan	male	2006-02-03	09264241894	gapusan@gmail.com	NULL	NULL	NULL	NULL	NULL	NULL	NULL
☐ Edit Copy Delete	8	PAT-0003	hiyuki manzo	male	2003-09-24	09264241894	justin@gmail.com	NULL	NULL	NULL	NULL	NULL	NULL	NULL
☐ Edit Copy Delete	9	PAT-0004	Jonathan Plazos	male	2004-06-23	098765432198	jonathan@gmail.com	NULL	NULL	NULL	NULL	NULL	NULL	NULL

[↑](#) ☐ Check all With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

IX. System Features and Functionality

The St. Gabriel Medical Center Queue Management System is organized to have eleven functional modules that address the specific operational needs of the platform, covering both the administrative and patient-facing perspectives.

1. Login

The Login module serves as the entry point of the St. Gabriel Medical Center Queue Management System. Registered users are required to provide their email address and password to authenticate and gain access to the system. The module also includes a Remember Me option for session persistence, a Forgot Password link for account recovery, and a redirect option for patients who do not yet have an account through the Create an Account link, ensuring a secure and accessible authentication experience.

2. Patient Registration

The Patient Registration module allows new patients to create an account on the system through a structured registration form. The patient is required to provide their first name, last name, email address, password, date of birth, gender, phone number, and home address. Upon successful registration, the patient account is created and the user is automatically logged in and redirected to the dashboard. The system assigns a unique patient code to each registered patient for identification purposes.

3. Dashboard

The Dashboard serves as the central overview of the platform's activity. For administrators and staff, it displays key statistics such as Total Patients, Today's Appointments, Waiting in Queue, and Now Serving. It also includes a Weekly Queue Activity bar chart that dynamically displays the total number of patient queues per day for the current week from Monday to Sunday, as well as a Quick Actions panel for navigating to frequently used functions and a Today's Queue preview section.

4. Patient Queue Management

The Patient Queue module handles the real-time management of patient queues across all departments. Administrators and staff can view the queue list per department using a department dropdown filter, call the next waiting patient, add walk-in patients to the queue, and skip patients when necessary. The queue list updates automatically every five seconds using AJAX polling without requiring a full page reload. Patients can view their own queue number, position, and overall queue status across all departments in a simplified combined view without the department filter.

5. Appointment Booking

The Appointment Booking module allows both patients and staff to schedule doctor visits through a structured four-step process. The user selects a department, then a doctor, then a date, and finally an available time slot. Available time slots are dynamically loaded via AJAX based on the selected doctor's schedule and existing bookings. An optional reason for visit

field is also provided. The system prevents double booking of the same doctor, date, and time slot at both the application and database trigger levels.

6. Appointment Management

The Appointment Management module allows administrators and staff to view all appointments across the system, while patients can only view their own. Appointments can be confirmed or canceled directly from the list or detail view. The system automatically marks appointments as confirmed when a patient joins the queue and as completed when their queue entry is marked as done, through database triggers.

7. Doctor Management

The Doctor Management module allows administrators and staff to view all registered doctors, add new doctors with their department, specialization, license number, and contact details. Doctors can be soft deleted, which deactivates all their schedules and cancels upcoming appointments while preserving all data in the database. Deleted doctors are listed in a Deleted Doctors section where they can be restored at any time.

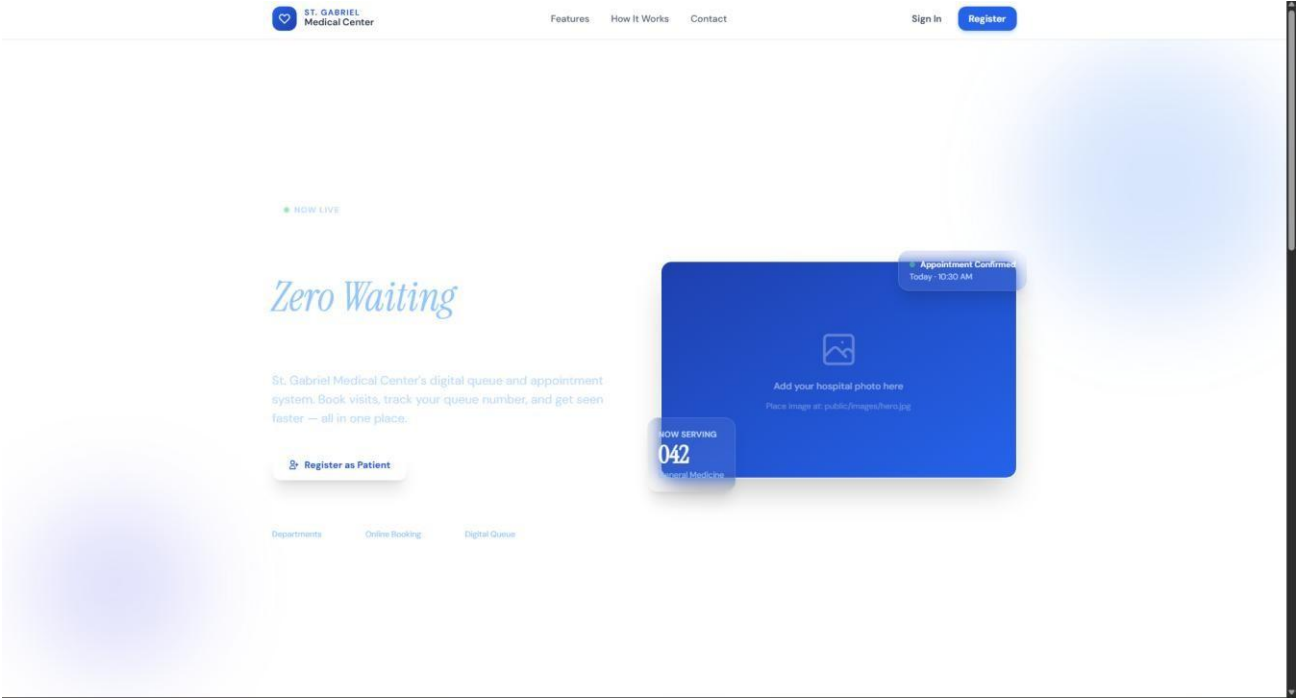
8. Doctor Schedule Management

The Doctor Schedule Management module allows administrators and staff to manage the weekly availability of each doctor. Schedules can be added per day of the week with a defined start time, end time, slot duration in minutes, and maximum number of patients. Schedules can be deactivated and restored without losing data. The system validates that the end time is always after the start time at the database trigger level.

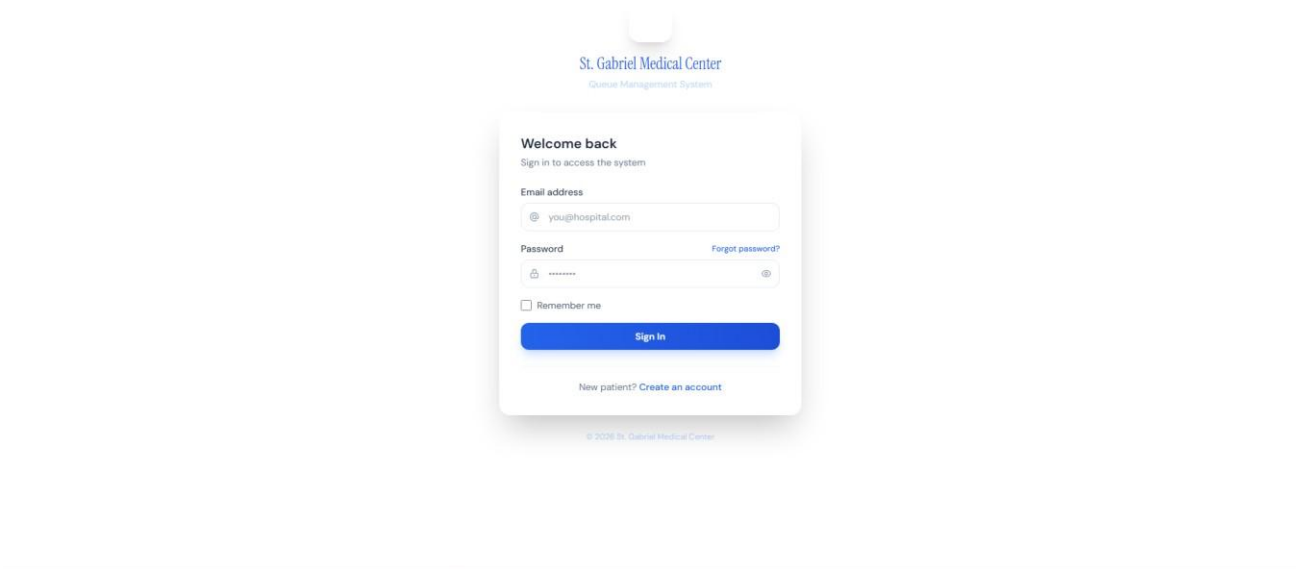
9. Patient Management

The Patient Management module allows administrators and staff to view all registered patients, search by name or patient code, view individual patient profiles including their appointment and queue history, and edit patient information. Administrators can soft delete a patient account, which moves it to the Deleted Patients list where it can be restored or permanently archived. Archived patients are stored in a separate archive table with a full data snapshot including appointment and queue history for audit purposes.

X. **Prototype / User Interface**



Landing Page



Login Page

St. Gabriel Medical Center

Queue Management System

New Patient Registration

Fill in the patient's personal information

PERSONAL INFORMATION

First Name *

Last Name *

Juan

Dela Cruz

Date of Birth *

Gender *

mm/dd/yyyy

Select gender...

Phone Number (optional)

0917034567

Address (optional)

House No., Street, Barangay, City

ACCOUNT CREDENTIALS

Email Address *

patient@example.com

Password *

Confirm Password *

Min. 8 characters

Re-enter password

Cancel

Register Patient

© 2020 St. Gabriel Medical Center

Register Page

ST. GABRIEL
Medical Center

Jonathan Plazos
Patient

MAIN

Dashboard

Patient Queue

Appointments

Book Appointment

My Profile

Dashboard

Tuesday, May 19, 2020

01:21:45 AM

Good afternoon,
Jonathan Plazos

Here's what's happening at the clinic today.

All time

4

Total Patients

Today

0

Appointments

Live

0

Waiting in Queue

None

—

Now Serving

Today's Queue

View all

1 Jonathan Plazos
Orthopedics - ORT-001

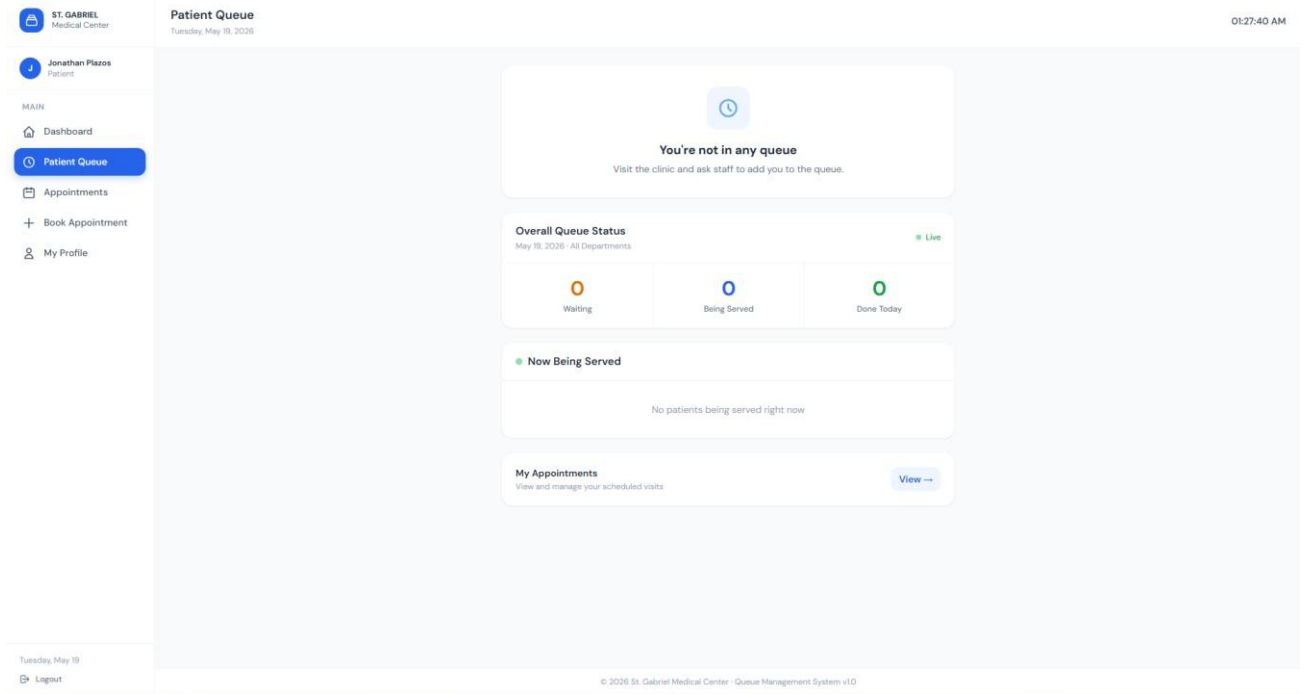
Skipped

Tuesday, May 19

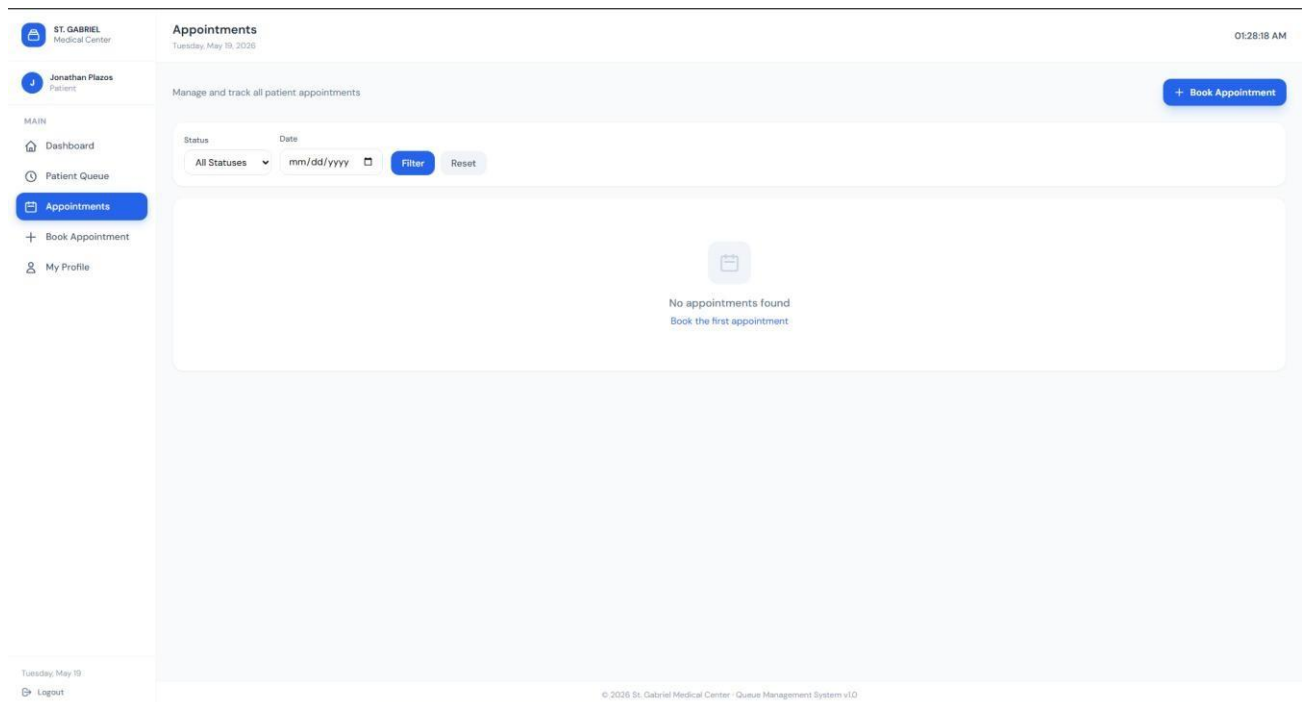
Login

© 2020 St. Gabriel Medical Center - Queue Management System v1.0

Home Page of Patient



Queue Page of Patient



Appointment Page of Patient

ST. GABRIEL
Medical Center

Jonathan Plazos
Patient

MAIN

Dashboard

Patient Queue

Appointments

+ Book Appointment

My Profile

Book Appointment

Tuesday, May 19, 2026

Tuesday, May 19

Logout

Fill in the details below to schedule an appointment.

My Appointments

New Appointment

St. Gabriel Medical Center

1 Select Department *

Choose a department...

2 Select Doctor *

Select a department first...

3 Select Date *

mm/dd/yyyy

4 Select Time Slot *

Select a doctor and date first

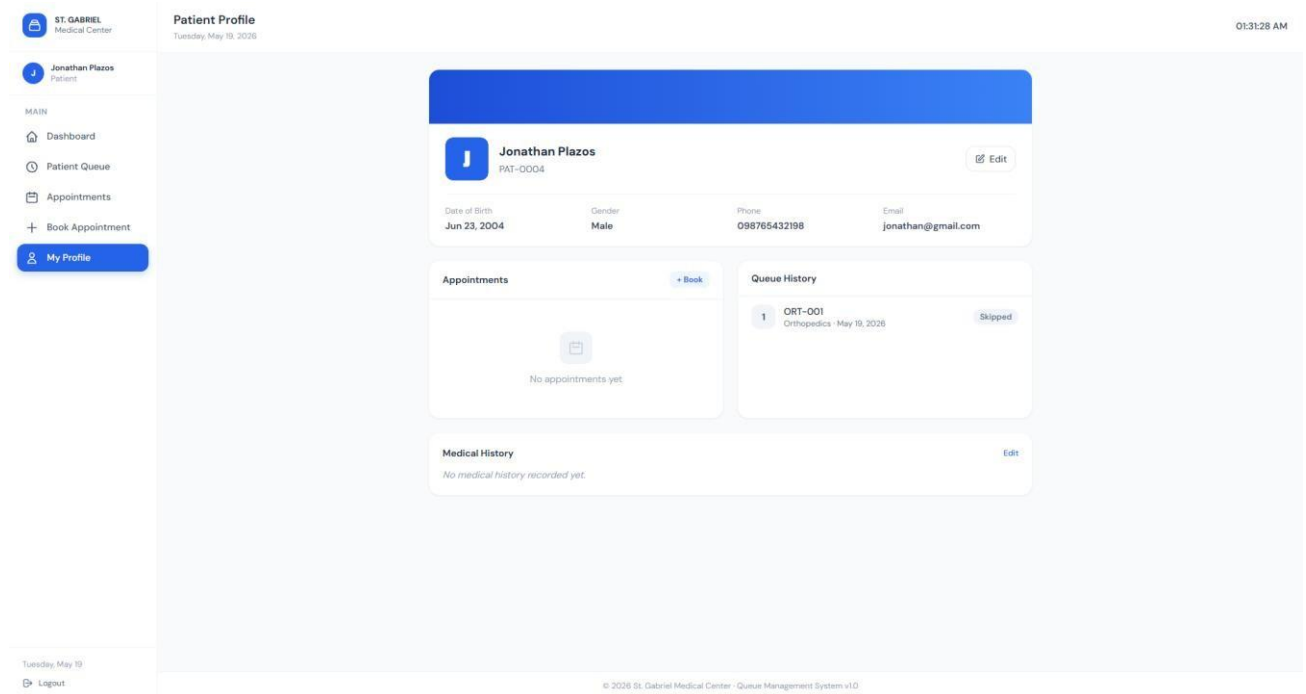
Reason for Visit (optional)

Briefly describe your symptoms or reason for the visit...

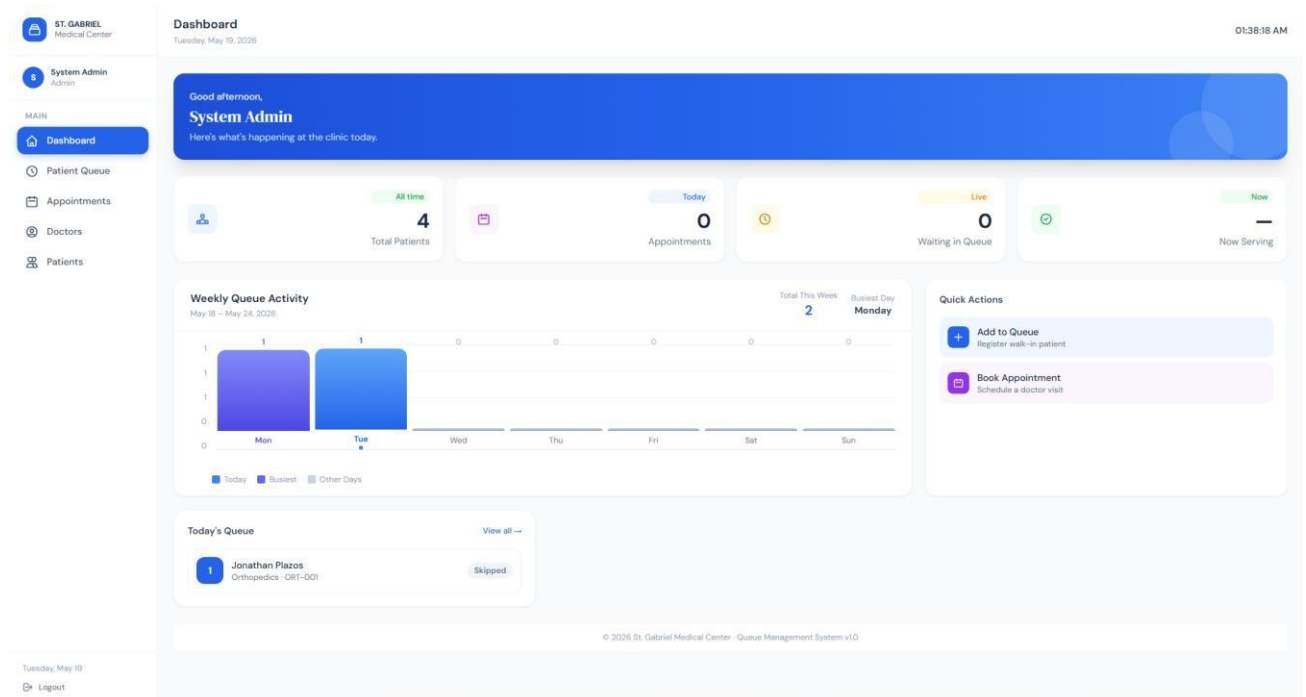
Book Appointment

© 2026 St. Gabriel Medical Center - Queue Management System v1.0

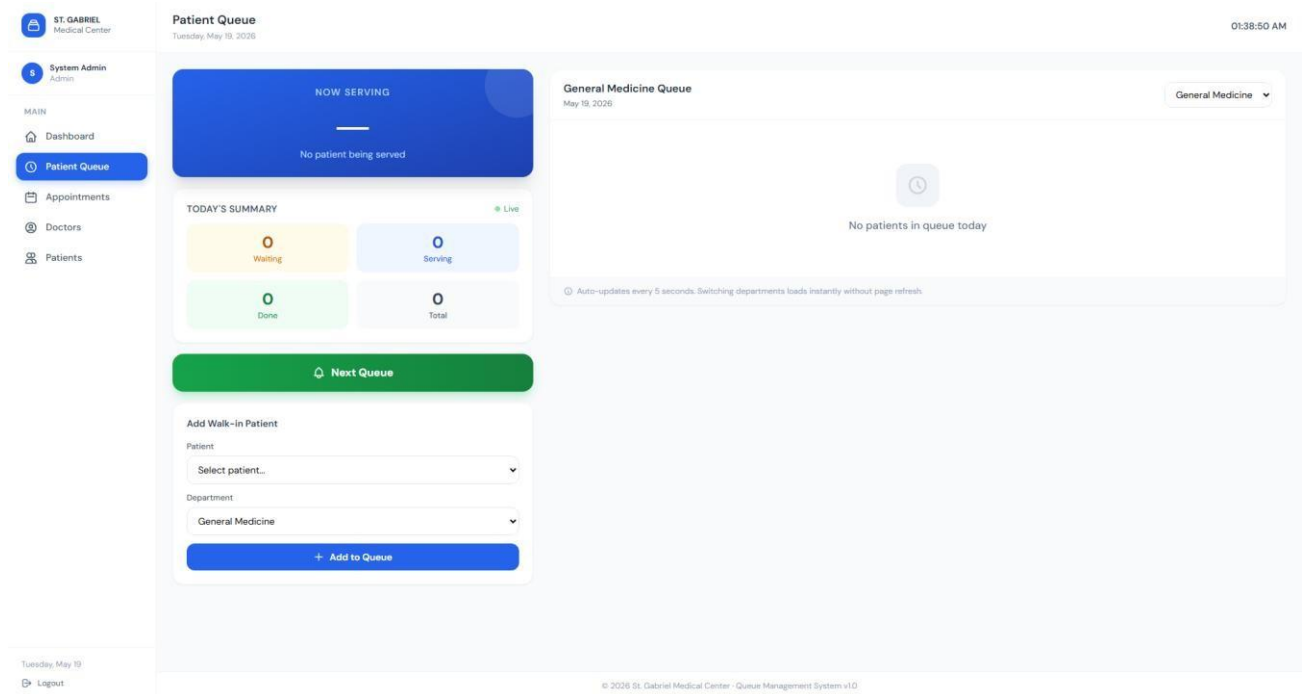
Book an Appointment patient



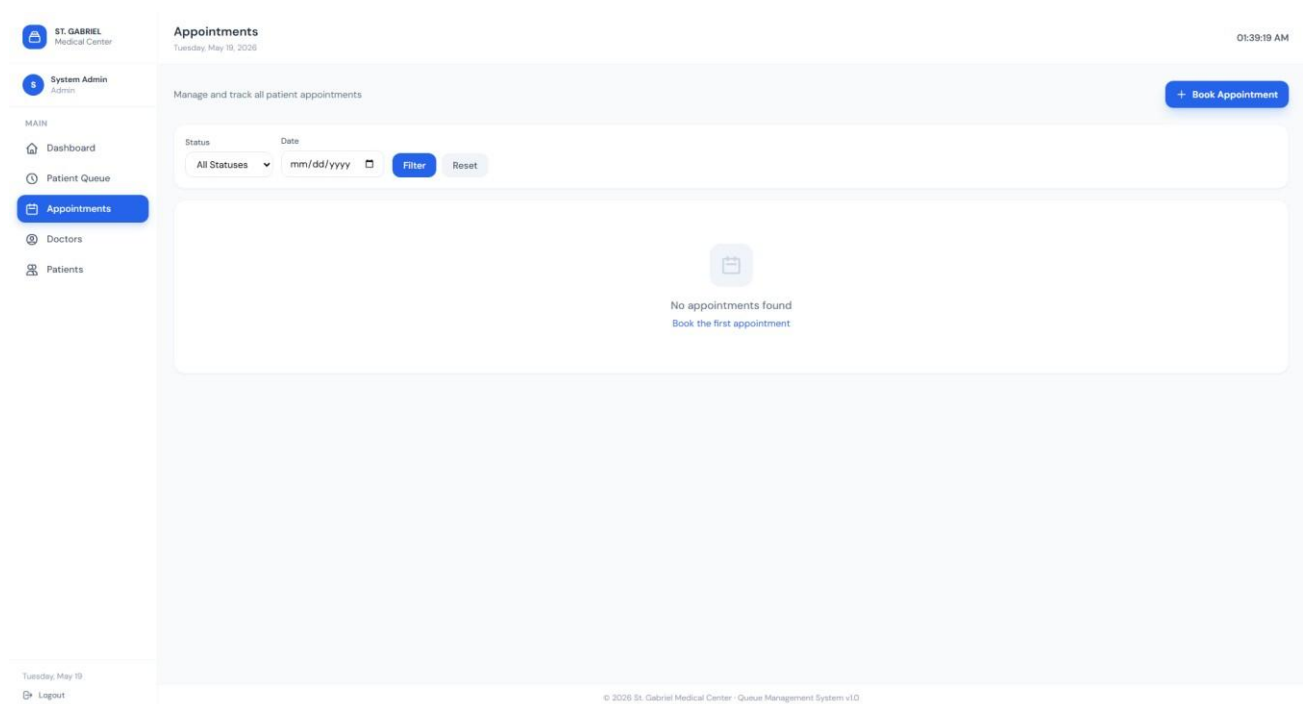
Patient Profile



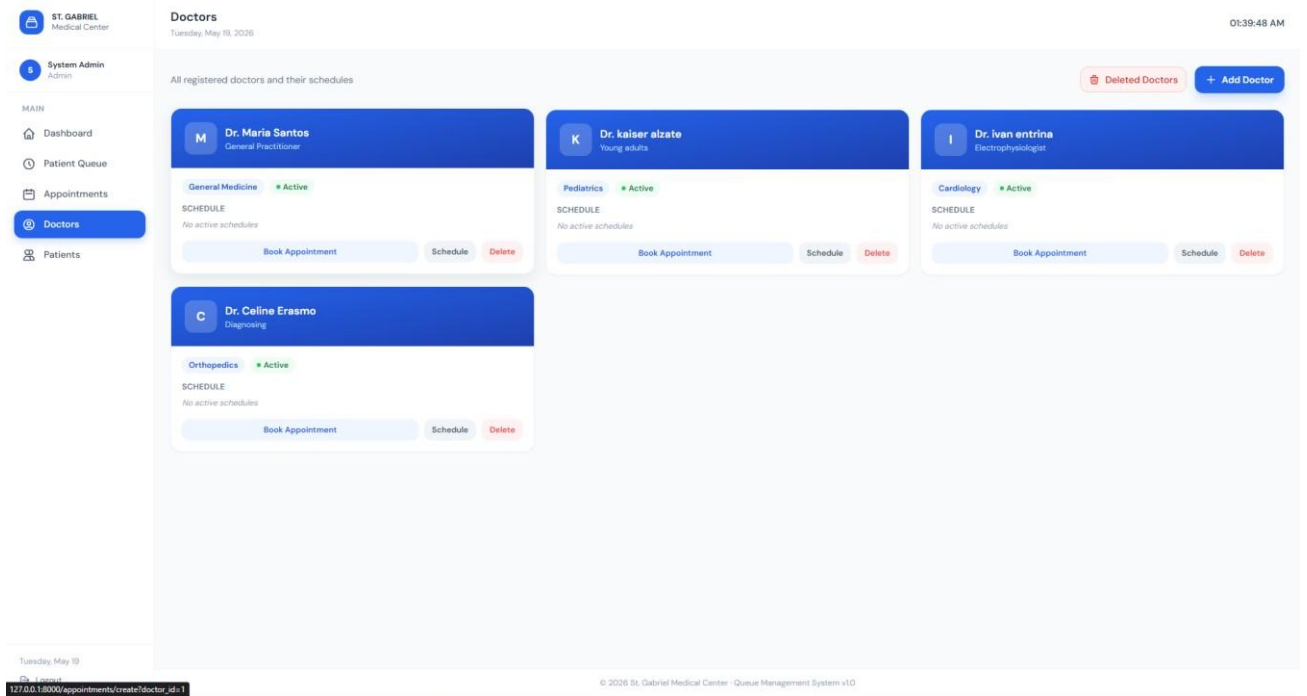
Admin Panel/Dashboard Page



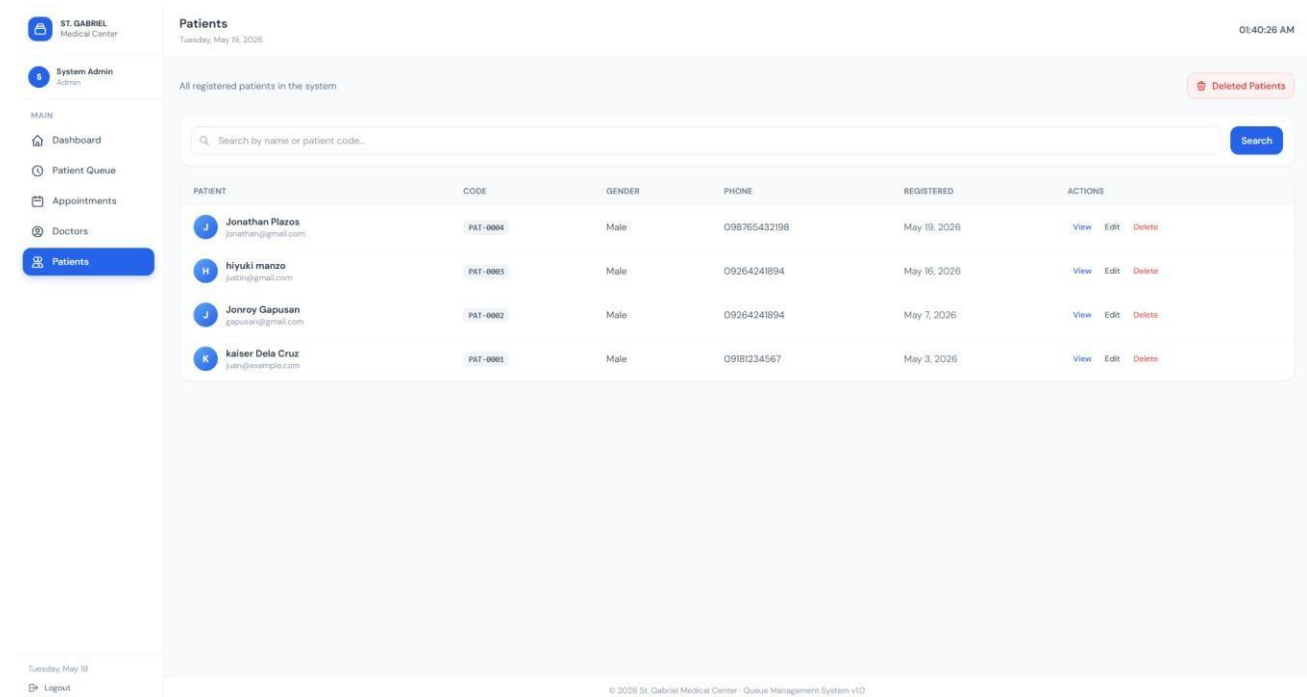
Admin Panel/Queue Management Page



Admin Panel/Appointment Page



Admin Panel/Doctor Management Page



Admin Panel/Patient Page

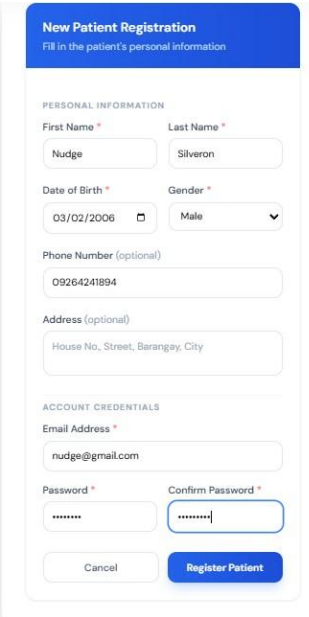
XI. Testing and Results

The CMS-Anime Blog system underwent several phases of testing to ensure data integrity, security, and a seamless user experience:

CRUD Functional Testing: Performed rigorous testing on the Create Post, Edit Post, Home Page, Post Read modules for the other users to see the post.

Testing (CRUD Post):

Register a Patient



The image shows a mobile application interface for a 'New Patient Registration' form. The form is titled 'New Patient Registration' with a subtitle 'Fill in the patient's personal information'. It is divided into two main sections: 'PERSONAL INFORMATION' and 'ACCOUNT CREDENTIALS'. The 'PERSONAL INFORMATION' section includes fields for 'First Name' (filled with 'Nudge'), 'Last Name' (filled with 'Silveron'), 'Date of Birth' (filled with '03/02/2006' and a calendar icon), 'Gender' (filled with 'Male' and a dropdown arrow), 'Phone Number (optional)' (filled with '09264241894'), and 'Address (optional)' (filled with 'House No., Street, Barangay, City'). The 'ACCOUNT CREDENTIALS' section includes fields for 'Email Address' (filled with 'nudge@gmail.com'), 'Password' (filled with '*****'), and 'Confirm Password' (filled with '*****'). At the bottom of the form are two buttons: 'Cancel' and 'Register Patient'.

New Patient Registration
Fill in the patient's personal information

PERSONAL INFORMATION

First Name * Last Name *

Nudge Silveron

Date of Birth * Gender *

03/02/2006 Male

Phone Number (optional)

09264241894

Address (optional)

House No., Street, Barangay, City

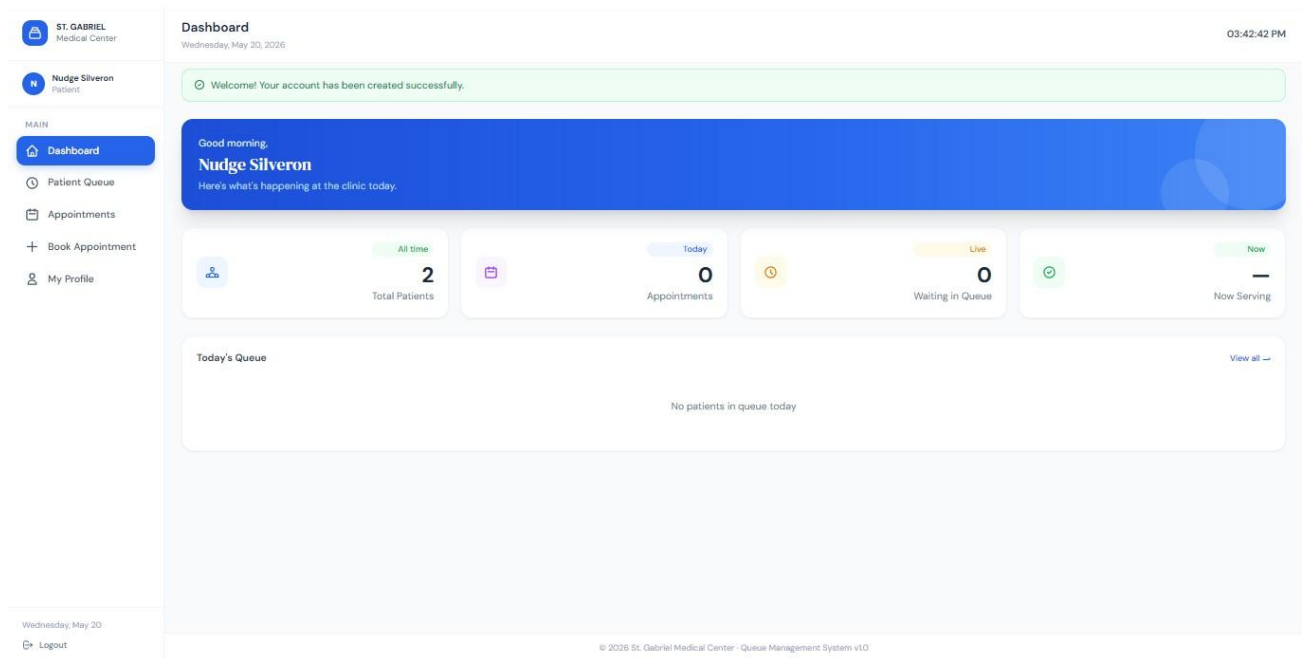
ACCOUNT CREDENTIALS

Email Address *

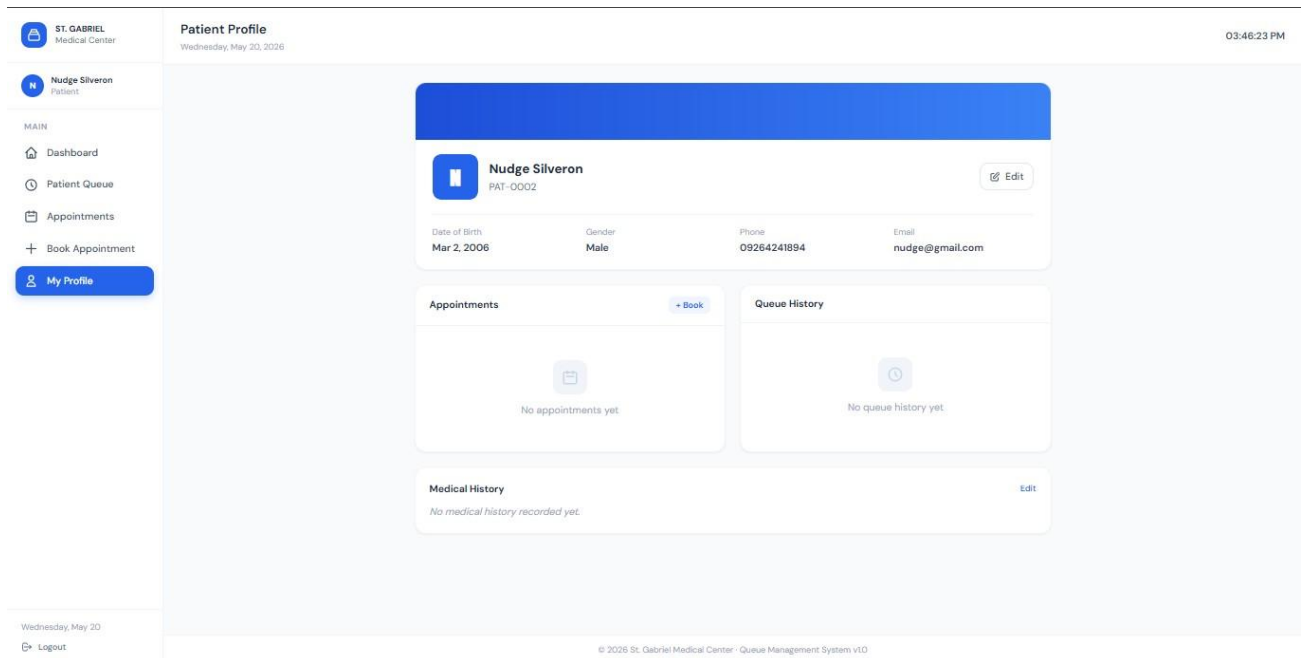
nudge@gmail.com

Password * Confirm Password *

Cancel Register Patient



Read Patient Profile Page



Edit Patient Profile

ST. GABRIEL
Medical Center

Nudge Silveron
Patient

MAIN

Dashboard

Patient Queue

Appointments

Book Appointment

My Profile

Edit Patient

Wednesday, May 20, 2026

03:47:47 PM

Edit Patient Profile

Nudge Silveron - PAT-0002

← Back

First Name *

Nudge

Last Name *

Silveron

Date of Birth *

03/02/2006

Gender *

Male

Phone

09264241894

Address

Medical History

Known conditions, allergies, past surgeries...

Cancel

Save Changes

Wednesday, May 20

Logout

© 2026 St. Gabriel Medical Center - Queue Management System v1.0

ST. GABRIEL
Medical Center

Liam Silveron
Patient

MAIN

Dashboard

Patient Queue

Appointments

Book Appointment

My Profile

Edit Patient

Wednesday, May 20, 2026

03:48:22 PM

Profile updated successfully.

Edit Patient Profile

Liam Silveron - PAT-0002

← Back

First Name *

Liam

Last Name *

Silveron

Date of Birth *

03/02/2006

Gender *

Male

Phone

09264241894

Address

Medical History

Known conditions, allergies, past surgeries...

Cancel

Save Changes

Wednesday, May 20

Logout

© 2026 St. Gabriel Medical Center - Queue Management System v1.0

ST. GABRIEL
Medical Center

Liam Silveron
Patient

MAIN

Dashboard

Patient Queue

Appointments

Book Appointment

My Profile

Patient Profile

Wednesday, May 20, 2026

03:48:40 PM

Liam Silveron

PAT-0002

Edit

Date of Birth

Mar 2, 2006

Gender

Male

Phone

09264241894

Email

nudge@gmail.com

Appointments

+ Book

No appointments yet

Queue History

No queue history yet

Medical History

Edit

No medical history recorded yet.

Wednesday, May 20

Logout

© 2026 St. Gabriel Medical Center - Queue Management System v1.0

Testing/ Admin View

ST. GABRIEL
Medical Center

System Admin
Admin

MAIN

Dashboard

Patient Queue

Appointments

Doctors

Patients

Wednesday, May 20

Logout

Patients

Wednesday, May 20, 2026

03:52:56 PM

All registered patients in the system

Deleted Patients

Search by name or patient code...

Search

PATIENT	CODE	GENDER	PHONE	REGISTERED	ACTIONS
J Juan Dela Cruz juan@example.com	PAT-0001	Male	09181234567	May 20, 2026	View Edit Delete

© 2026 St. Gabriel Medical Center - Queue Management System v1.0

ST. GABRIEL
Medical Center

System Admin
Admin

MAIN

Dashboard

Patient Queue

Appointments

Doctors

Patients

Wednesday, May 20

Logout

Deleted Patients

Wednesday, May 20, 2026

03:53:41 PM

Soft-deleted patients. Restore to make active again, or Archive to permanently save the record.

View Archives

Active Patients

Restore — brings the patient back as active. | Archive — saves a permanent snapshot then permanently removes the record.

PATIENT	CODE	GENDER	PHONE	DELETED AT	ACTIONS
L Liam Silveron Deleted	PAT-0002	Male	09264241894	May 20, 2026 07:52 AM	Restore Archive

© 2026 St. Gabriel Medical Center - Queue Management System v1.0

XII. Conclusion and Recommendations

Conclusion

The St. Gabriel Medical Center Queue Management System is a web-based platform developed using the Laravel framework, designed to serve as a dedicated system for managing patient queues, appointments, doctors, and patient records within a hospital or clinic environment. The system was built with two distinct perspectives in mind, which are the administrative side and the patient-facing side, each equipped with a dedicated set of modules that address their respective roles and responsibilities within the platform.

On the administrative side, the platform provides a comprehensive management panel that gives administrators and staff full control over the clinic's daily operations. Through modules such as the Dashboard, Patient Queue Management, Appointment Management, Doctor Management, Doctor Schedule Management, and Patient Management, administrators and staff are equipped with the necessary tools to monitor real-time queue activity, manage patient records, oversee appointment schedules, and maintain doctor availability. These modules collectively ensure that the clinic operates smoothly, efficiently, and in alignment with its intended operational standards. The inclusion of a Weekly Queue Activity analytics chart further empowers administrators to monitor patient flow trends across the current week and make informed decisions regarding staffing and resource allocation.

On the patient-facing side, the platform delivers a clear and accessible experience for registered patients. Starting from a straightforward registration process, patients are guided through setting up their account before gaining access to features such as the Dashboard, Patient Queue Status, Appointment Booking, and Profile Management. These modules empower patients to monitor their queue position in real time, book appointments with their preferred doctor, and manage their personal health information with ease. The patient interface is intentionally simplified, removing unnecessary administrative controls while providing all the information a patient needs during their clinic visit.

Overall, the St. Gabriel Medical Center Queue Management System successfully addresses the core operational needs of a hospital queue management environment by combining robust administrative controls with a patient-friendly interface. The system demonstrates a well-structured approach to queue management, appointment scheduling, doctor administration, and patient record keeping, making it a functional and purposeful web application for its target healthcare institution.

Recommendations

While the St. Gabriel Medical Center Queue Management System has successfully implemented its core features and functionalities, there are several areas where the system can be further improved and expanded to enhance both the patient experience and administrative efficiency. The following recommendations are proposed for future development:

1. SMS and Email Notification System

Integrating an SMS and email notification feature would ensure that patients are informed of important updates even when they are not actively logged into the system, covering events such as appointment confirmations, queue number calls, appointment reminders, and cancellation notices. This would significantly reduce patient wait time confusion and missed appointments.

2. WebSocket-Based Real-Time Updates

While the current system uses AJAX polling every five seconds for real-time queue updates, implementing WebSocket technology through Laravel Echo and Pusher would provide truly instantaneous updates with lower server load. This would be particularly beneficial during high-traffic periods when many patients are simultaneously monitoring their queue status.

3. Appointment Cancellation and Rescheduling by Patients

Allowing patients to cancel or reschedule their own appointments directly from the patient portal would reduce the administrative burden on staff and give patients greater control over their healthcare scheduling without requiring them to contact the clinic directly.

4. Doctor Availability Calendar View

Implementing a visual calendar interface for viewing doctor schedules and appointment slots would make it significantly easier for both staff and patients to identify available time slots at a glance, reducing the time spent navigating the current dropdown-based slot selection system.

5. Patient Medical History and Records Management

Expanding the patient profile module to include a more comprehensive medical history section, allowing doctors and staff to record diagnoses, prescriptions, and visit notes directly within the system, would transform the platform from a queue management tool into a more complete clinic management solution.

6. Reporting and Analytics Export

Adding the ability to export queue statistics, appointment summaries, and patient records as PDF or Excel files would allow administrators to generate formal reports for management review, auditing, and regulatory compliance purposes, extending the usefulness of the existing analytics dashboard.

7. Mobile Application

Developing a dedicated mobile application for both Android and iOS platforms would allow patients to monitor their queue position, receive push notifications, and book appointments directly from their smartphones without needing to access the web-based interface, significantly improving the overall patient experience.